

State Management Differences with PWA Kit

PWA Kit's state management is React Query-centric, treating the API as the source of truth with sophisticated caching. Storefront Next takes a server-first approach where loaders provide data and client state is minimal and feature-specific. The shift eliminates the need for a dedicated server state library, simplifying the mental model while enabling better streaming and Suspense integration.

Contents

- [Key Differences](#)
- [Server State Management](#)
- [Key Similarities](#)
- [Storefront Conversion Considerations](#)

Key Differences

Aspect	PWA Kit	Storefront Next
Primary Approach	React Query for server state	React Router loaders
State Library	React Query + Context	React Router Context + React State
Caching	React Query in-memory cache	Router revalidation
Server State	Managed by React Query	Managed by React Router
Persistence	localStorage	Cookies
Architecture	Centralized (React Query)	Decentralized

Server State Management

PWA Kit uses React Query for all server state:

```
</> 1 // PWA Kit: Data fetched in component via hooks
2 const ProductPage = ({ productId }) => {
3   const { data, isLoading } = useProduct({ productId })
```



```
1 // Storefront Next: Data fetched before component renders
2 export function loader({ params, context }) {
3   return { product: fetchProduct(params.productId) };
4 }
5
6 const ProductPage = ({ loaderData }) => {
7   const product = use(loaderData.product); // Data is available here
8 };
```

Key Similarities

1. React Context for distribution: Both use Context to make state available to components.
2. Provider pattern: Both wrap the app in provider components.
3. Hooks for consumption: Both expose custom hooks (useAuth, useBasket, etc.).
4. Persistence considerations: Both handle localStorage/cookies for persistence.
5. SSR awareness: Both handle server vs client rendering differences.

Storefront Conversion Considerations

Keep these considerations in mind when converting your storefront from PWA Kit to Storefront Next.

- React Query to Loaders
 - Move data fetching from component hooks to route loaders.
 - Replace `isLoading` checks with Suspense boundaries.
 - Remove React Query configuration and providers.
- Context Simplification
 - Simplify contexts to just hold values (no complex logic).
 - Move complex state to Zustand if needed.
 - Use middleware for initialization instead of provider effects.
- State Persistence
 - Migrate localStorage usage to cookies for SSR compatibility.
 - Use middleware to read persisted state during request processing.
- Derived State



Try for free



**DID THIS ARTICLE SOLVE YOUR
ISSUE?**

Let us know so we can improve!

[Share your feedback](#)



DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

COMMUNITY

Trailblazer Community
Events and Calendar
Partner Community
Blog
Salesforce Admins
Salesforce Architects

© Copyright 2026 Salesforce, Inc. [All rights reserved.](#) Various trademarks held by their respective owners. Salesforce, Inc.
Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)  [Your Privacy Choices](#)

